

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по учебной практике

УП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции
программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____ Олехнович Мария Павловна
(подпись) (фамилия, имя, отчество)

Группы ИПо-32

Руководитель практической подготовки от профильной организации

Преподаватель _____
(должность)

_____ Берестов Дмитрий Борисович
(подпись) (фамилия, имя, отчество)

МП «26» июня 2026 года

Руководитель практической подготовки от института

_____ Берестов Дмитрий Борисович
(подпись) (фамилия, имя, отчество)

«27» июня 2026 года

Пермь, 2026 год

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
РАЗДЕЛ 1. РАЗРАБОТКА ТРЕБОВАНИЯ К ПРОГРАММНЫМ МОДУЛЯМ НА ОСНОВЕ АНАЛИЗА ПРОЕКТНОЙ И ТЕХНИЧЕСКОЙ ДОКУМЕНТАЦИИ НА ПРЕДМЕТ ВЗАИМОДЕЙСТВИЯ КОМПОНЕНТ.....	5
1.1. Сбор информации о существующем состоянии продукта.	5
1.2. Разработка технического задания.	6
РАЗДЕЛ 2. ИНТЕГРАЦИЯ МОДУЛЕЙ В ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ	8
2.1. Процесс разработки программного обеспечения	8
2.2. Инструменты и среда программирования.....	9
2.3. Диаграмма классов	10
2.4. Разработка программного модуля.....	12
2.5. Разработка интерфейса	15
РАЗДЕЛ 3. ОТЛАДКА ПРОГРАММНОГО МОДУЛЯ	С
ИСПОЛЬЗОВАНИЕМ СПЕЦИАЛИЗИРОВАННЫХ ПРОГРАММНЫХ СРЕДСТВ	17
3.1. Процесс отладки.	17
РАЗДЕЛ 4. ОСУЩЕСТВЛЯТЬ РАЗРАБОТКУ ТЕСТОВЫХ НАБОРОВ И ТЕСТОВЫХ СЦЕНАРИЕВ ДЛЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ	19
4.1. Процесс тестирования.....	19
4.2. Создание unit-тестов.....	19
5.1. Процесс инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования.....	22
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ	26
Приложение 1.....	27

ВВЕДЕНИЕ

Практика имеет целью комплексное освоение студентами всех видов профессиональной деятельности по специальности 09.02.07 «Информационные системы и программирование», формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности.

Учебная практика направлена на формирование у обучающихся первоначальных практических профессиональных умений в рамках модулей ППСЗ по основным видам профессиональной деятельности для освоения методов и приемов практического применения прикладных программных продуктов для программного обеспечения компьютерных систем

В настоящей работе представлен отчет о прохождении учебной практики студента-практиканта. Основными видами деятельности являлись: Разработка программного обеспечения; выполнение отладки программного модуля с использованием специализированных программных средств; осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения; уметь производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования

Задачи практики (освоение обучающимися профессиональных и общих компетенций):

1. Разрабатывать требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент;
2. Выполнять интеграцию модулей в программное обеспечение;
3. Выполнять отладку программного модуля с использованием специализированных программных средств;

4. Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения.
5. Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования
6. Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам;
7. Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности
8. Планировать и реализовывать собственное профессиональное и личностное развитие;
9. Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами
10. Осуществлять устную и письменную коммуникацию на государственном языке Российской Федерации с учетом особенностей социального и культурного контекста.
11. Использовать информационные технологии в профессиональной деятельности
12. Пользоваться профессиональной документацией на государственном и иностранном языках

РАЗДЕЛ 1. РАЗРАБОТКА ТРЕБОВАНИЯ К ПРОГРАММНЫМ МОДУЛЯМ НА ОСНОВЕ АНАЛИЗА ПРОЕКТНОЙ И ТЕХНИЧЕСКОЙ ДОКУМЕНТАЦИИ НА ПРЕДМЕТ ВЗАИМОДЕЙСТВИЯ КОМПОНЕНТ

1.1. Сбор информации о существующем состоянии продукта.

В рамках учебной практики необходимо разработать приложение для управления задачами с использованием языка программирования C# и платформы .NET Framework. Приложение должно обеспечивать полный цикл работы с задачами: от создания до архивирования, включая фильтрацию, поиск, сортировку и статистический анализ.

До разработки программного продукта управление задачами осуществлялось вручную или с помощью простых текстовых редакторов (Notepad, Word). Основные проблемы:

- отсутствие структурированного хранения;
- невозможность быстрого поиска;
- отсутствие приоритизации и фильтрации;
- риск потери данных.

Разрабатываемое приложение «Менеджер задач» должно решить все выявленные проблемы.

Сравнительный анализ процессов «AS IS» и «TO BE» (см. таб. 1)

Таблица 1. Сравнительный анализ

Параметр	AS IS (до)	TO BE (после)
Формат хранения	Текстовые файлы, хаотично	JSON-файл, структурированно
Добавление задачи	Ручной ввод в текстовый файл	Форма ввода с валидацией
Просмотр списка	Открытие файла, прокрутка	Таблица с сортировкой
Поиск	Ручной (Ctrl+F в блокноте)	Автоматический по названию/описанию

Фильтрация, сортировка, приоритет, важные задачи	Отсутствует	Присутствует и имеет свои обозначительные признаки
Тестирование	Отсутствует	Модульные тесты (MSTest)

1.2. Разработка технического задания.

Техническое задание разработано в соответствии с требованиями ГОСТ 19.201-78 «ЕСПД. Техническое задание. Требования к содержанию и оформлению» и ГОСТ 34.602-89 «Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы».

Стандарты, используемые для написания технического задания

1. ГОСТ 19.101-77 - ЕСПД. Виды программ и программных документов
2. ГОСТ 19.201-78 - ЕСПД. Техническое задание. Требования к содержанию и оформлению
3. ГОСТ 19.401-78 - ЕСПД. Текст программы. Требования к содержанию и оформлению
4. ГОСТ 19.404-79 - ЕСПД. Пояснительная записка. Требования к содержанию и оформлению
5. ГОСТ 34.602-89 - Техническое задание на создание АС

Целью разработки является создание приложения для эффективного управления личными и рабочими задачами с возможностью приоритизации, отслеживания статуса и статистического анализа.

Структура технического задания:

1. Общие сведения
 - Наименование: Менеджер задач
 - Язык программирования: C#
 - Платформа: .NET Framework 4.6
 - Среда разработки: Microsoft Visual Studio 2022
 - Тип приложения: Windows Forms (WinForms)
2. Функциональные требования
 - Добавление новой задачи (название, описание, приоритет, срок выполнения, статус)

- Просмотр списка всех задач в табличном виде
- Фильтрация задач по статусу (Новая, В процессе, Завершена)
- Поиск задач по названию или описанию
- Редактирование существующих задач (изменение статуса)
- Удаление задач
- Сохранение и загрузка задач в/из JSON-файла
- Сортировка по приоритету и сроку выполнения
- Пометка задач как «Важная» (визуальное выделение)
- Отображение статистики (всего, завершено, просрочено)

3. Нефункциональные требования

- Время отклика интерфейса: не более 1 секунды
- Формат хранения данных: JSON (UTF-8)
- Автоматическое сохранение при каждой операции
- Архитектура: разделение на DLL (логика) и WinForms (UI)

4. Требования к составу и параметрам технических средств

- Операционная система: Windows 7 и выше
- Процессор: 1 ГГц и выше
- Оперативная память: 512 МБ и выше
- Свободное место на диске: 50 МБ

5. Требования к программной документации

- Пояснительная записка
- Диаграмма классов
- Исходный код с комментариями
- Модульные тесты

РАЗДЕЛ 2. ИНТЕГРАЦИЯ МОДУЛЕЙ В ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

2.1. Процесс разработки программного обеспечения

Для разработки программного продукта была выбрана итерационная модель разработки.

Обоснование выбора:

- Проект имеет чётко определённые требования, но требует постепенного наращивания функционала (сначала базовые операции CRUD, затем фильтрация, поиск, сортировка, статистика);
- Возможность быстрого получения работающего прототипа и его демонстрации руководителю практики;
- Гибкость внесения изменений по ходу разработки (например, переход от WPF к WinForms по требованию);
- Простота тестирования каждой итерации отдельно (сначала тесты для библиотеки, затем для UI);
- Учебный характер проекта позволяет экспериментировать и учиться на ошибках без риска для бизнеса.

Этапы итерационной разработки:

Итерация 1. Создание структуры проекта (3 проекта: Core, Tests, WinForms), базовая модель TaskItem, интерфейс ITaskService;

Итерация 2. Реализация TaskService с операциями CRUD (Add, Get, Update, Delete), JSON-сериализация через Newtonsoft.Json;

Итерация 3. Добавление фильтрации, поиска, сортировки с использованием LINQ;

Итерация 4. Разработка Windows Forms интерфейса с элементами управления (TextBox, ComboBox, ListView, Button);

Итерация 5. Реализация статистики, визуального выделения важных и просроченных задач;

Итерация 6. Написание модульных тестов (MSTest), отладка, исправление ошибок совместимости .NET Framework 4.6.

2.2. Инструменты и среда программирования

Microsoft Visual Studio 2022 - Интегрированная среда разработки (IDE) для создания приложений на C#. Использовалась для написания кода, отладки, тестирования и сборки проекта.

.NET Framework 4.6 - Платформа для разработки и выполнения приложений. Обеспечивает совместимость с Windows Forms, LINQ, сериализацией JSON.

C# 6.0 - Язык программирования с поддержкой ООП, LINQ, коллекций, делегатов, событий.

Newtonsoft.Json 13.0.3 - Библиотека для сериализации и десериализации объектов в формат JSON. Выбрана вместо System.Text.Json из-за совместимости с .NET Framework 4.6.

MSTest (Microsoft.VisualStudio.TestTools) - Фреймворк для модульного тестирования. Встроен в Visual Studio, не требует дополнительной установки.

NuGet Package Manager - Менеджер пакетов для установки сторонних библиотек (Newtonsoft.Json).

Среда программирования:

Для разработки использовалась среда Microsoft Visual Studio 2022 Community Edition. Среда предоставляет следующие возможности:

- IntelliSense - автодополнение кода, подсказки по методам и свойствам;
- Отладчик - пошаговое выполнение, точки останова, просмотр значений переменных;
- Дизайнер форм - визуальное проектирование интерфейса Windows Forms перетаскиванием элементов;
- Обзорщик решений - управление проектами, файлами, ссылками;
- Тестовый обзорщик - запуск и анализ результатов модульных тестов;

- NuGet Package Manager - установка и управление пакетами (Newtonsoft.Json).

2.3. Диаграмма классов

Диаграмма классов (Class Diagram) описывает статическую структуру системы, показывая классы, их атрибуты, методы и отношения между ними.

1. Класс TaskItem (Модель задачи)

- Пространство имён: TaskManager.Core.Models
- Назначение: Представляет сущность «Задача» с полным набором свойств.

Свойства:

- Id (Guid) - уникальный идентификатор задачи, генерируется автоматически;
- Title (string) - название задачи;
- Description (string) - описание задачи;
- Priority (TaskPriority) - приоритет: Low, Medium, High;
- DueDate (DateTime) - срок выполнения;
- Status (TaskStatus) - статус: New, InProgress, Completed;
- IsImportant (bool) - флаг «Важная задача»;
- CreatedAt (DateTime) - дата создания, устанавливается автоматически;
- IsOverdue (bool, вычисляемое) - true, если срок прошёл и статус не Completed.

2. Перечисления:

- TaskStatus: New - новая задача; InProgress - в процессе выполнения; Completed - завершённая.
- TaskPriority: Low - низкий приоритет; Medium - средний приоритет; High - высокий приоритет.

3. Интерфейс ITaskService

- Пространство имён: TaskManager.Core.Services

- Назначение: определяет контракт (договор) для сервиса управления задачами. Реализация паттерна «Репозиторий».

Методы:

- AddTask(string title, string description, TaskPriority priority, DateTime dueDate, bool isImportant) - добавление задачи;
- GetAllTasks() - получение всех задач;
- GetTasksByStatus(TaskStatus status) - фильтрация по статусу;
- SearchTasks(string query) - поиск по названию/описанию;
- GetTaskById(Guid id) - получение задачи по ID;
- UpdateTask(TaskItem task) - обновление задачи;
- DeleteTask(Guid id) - удаление задачи;
- SortByPriority() - сортировка по приоритету (убывание);
- SortByDueDate() - сортировка по сроку (возрастание);
- GetStatistics() - получение статистики (Tuple<int, int, int>);
- SaveToFile(string filePath) - сохранение в JSON;
- LoadFromFile(string filePath) - загрузка из JSON.

4. Класс TaskService

- Пространство имён: TaskManager.Core.Services
- Наследование: реализует интерфейс ITaskService
- Назначение: основная бизнес-логика приложения. Управляет коллекцией задач в памяти и синхронизирует её с JSON-файлом.

Поля:

- _tasks (List<TaskItem>) - коллекция задач в оперативной памяти;
- _filePath (string) - путь к файлу JSON.

Ключевые особенности реализации:

- Использование LINQ для фильтрации, поиска, сортировки;
- Автоматическое сохранение при каждой операции модификации;
- Десериализация при создании сервиса (если файл существует).

5. Класс Form1 (Windows Forms)

- Пространство имён: TaskManager.WinForms

- Наследование: System.Windows.Forms.Form
- Назначение: Графический пользовательский интерфейс приложения.

Поля:

- `_taskService (ITaskService)` - ссылка на сервис задач;
- `_selectedTask (TaskItem)` - текущая выбранная задача.

Элементы управления:

- `txtTitle, txtDescription` - поля ввода названия и описания;
- `cmbPriority` - выпадающий список приоритетов;
- `dtpDueDate` - выбор даты срока выполнения;
- `chkImportant` - флажок «Важная»;
- `btnAdd, btnDelete, btnInProgress, btnCompleted` - кнопки управления;
- `txtSearch, btnSearch` - поле и кнопка поиска;
- `cmbFilter, btnFilter` - фильтр по статусу;
- `btnSortPriority, btnSortDate` - кнопки сортировки;
- `lblStats` - метка статистики;
- `lvTasks` - таблица (ListView) отображения задач.

2.4. Разработка программного модуля

Разработка программного модуля осуществлялась поэтапно в соответствии с итерационной моделью. Ниже описаны ключевые этапы с примерами кода.

Этап 1. Создание модели данных (TaskItem.cs). Создан класс TaskItem, представляющий сущность «Задача». Используются автоматические свойства, вычисляемое свойство IsOverdue и конструктор по умолчанию для инициализации Id и CreatedAt.

```
using System;

namespace TaskManager.Core.Models
{
    public enum TaskStatus
    {
        New,
```

```

        InProgress,
        Completed
    }

    public enum TaskPriority
    {
        Low,
        Medium,
        High
    }

    public class TaskItem
    {
        public Guid Id { get; set; } = Guid.NewGuid();
        public string Title { get; set; } = string.Empty;
        public string Description { get; set; } = string.Empty;
        public TaskPriority Priority { get; set; }
        public DateTime DueDate { get; set; }
        public TaskStatus Status { get; set; }
        public bool IsImportant { get; set; }
        public DateTime CreatedAt { get; set; } = DateTime.Now;

        public bool IsOverdue => Status != TaskStatus.Completed && DueDate <
DateTime.Now;
    }
}

```

Этап 2. Определение интерфейса (ITaskService.cs). Интерфейс ITaskService определяет контракт для всех операций с задачами. Использование интерфейса позволяет применять принцип инверсии зависимостей (DIP) и легко заменять реализацию.

```

using System;
using System.Collections.Generic;
using TaskManager.Core.Models;

namespace TaskManager.Core.Services

```

```

{
    public interface ITaskService
    {
        TaskItem AddTask(string title, string description, TaskPriority priority,
            DateTime dueDate, bool isImportant = false);
        IEnumerable<TaskItem> GetAllTasks();
        IEnumerable<TaskItem> GetTasksByStatus(TaskManager.Core.Models.TaskStatus
status);
        IEnumerable<TaskItem> SearchTasks(string query);
        TaskItem GetTaskById(Guid id);
        bool UpdateTask(TaskItem task);
        bool DeleteTask(Guid id);
        IEnumerable<TaskItem> SortByPriority();
        IEnumerable<TaskItem> SortByDueDate();
        Tuple<int, int, int> GetStatistics();
        void SaveToFile(string filePath);
        void LoadFromFile(string filePath);
    }
}

```

Этап 3. Разработка Windows Forms интерфейса (Form1.cs). Графический интерфейс разработан с использованием Windows Forms. Реализованы все требования ТЗ: добавление, удаление, редактирование статуса, поиск, фильтрация, сортировка, отображение статистики.

```

using System;
using System.Drawing;
using System.Linq;
using System.Windows.Forms;
using TaskManager.Core.Models;
using TaskManager.Core.Services;

namespace TaskManager.WinForms
{
    public partial class Form1 : Form
    {
        private readonly ITaskService _taskService;
    }
}

```

```

private TaskItem _selectedTask;

public Form1()
{
    InitializeComponent();
    _taskService = new TaskService("tasks.json");
    SetupColumns();
    SetupPriorityCombo();
    SetupFilterCombo();
    RefreshTasks();
    UpdateStatistics();
}

...
private void UpdateStatistics()
{
    var stats = _taskService.GetStatistics();
    lblStats.Text = string.Format("Всего: {0} | Завершено: {1} | Просрочено: {2}",
        stats.Item3, stats.Item1, stats.Item2);
}
}
}

```

2.5. Разработка интерфейса

1. Проектирование. На этапе проектирования была разработана схема расположения элементов управления на форме:

- Верхняя часть: панель ввода данных (название, описание, приоритет, срок, флажок «Важная»)
- Средняя часть: кнопки управления (Добавить, Удалить, В процессе, Завершена)
- Правая часть: панель поиска, фильтрации и сортировки
- Нижняя часть: метка статистики и таблица ListView

2. Прототипирование. Прототип создан в визуальном дизайнера Visual Studio методом перетаскивания элементов из Toolbox. Настроены свойства:

имена (Name), тексты (Text), цвета (BackColor, ForeColor), привязки событий (Click, SelectedIndexChanged).

3. Стилизация:

- Кнопка «Добавить» — зелёный фон (Color.Green), белый текст
- Кнопка «Удалить» — красный фон (Color.Red), белый текст
- Важные задачи — жёлтый фон (Color.LightYellow)
- Просроченные задачи — красный фон (Color.LightCoral)
- Статистика — жирный шрифт, тёмно-красный цвет (Color.DarkRed)

Итоговый разработанный интерфейс можно ниже (см. рис. 1).

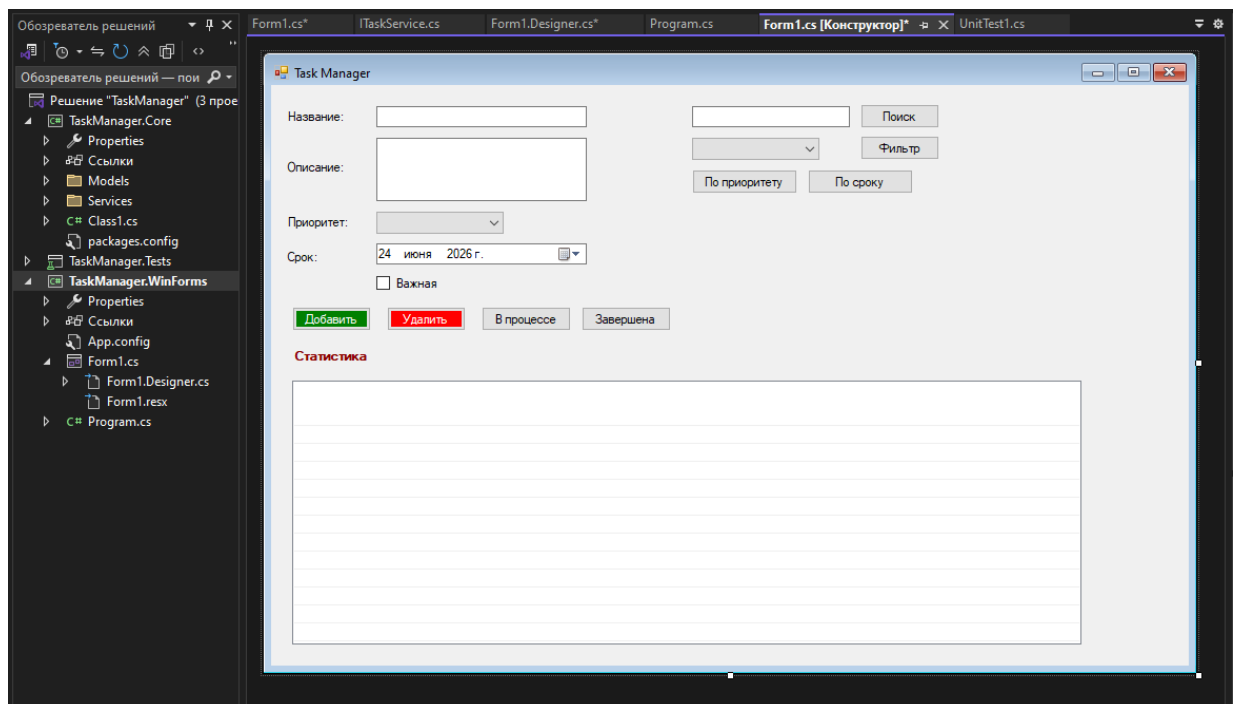


Рисунок 1. Итоговый интерфейс

РАЗДЕЛ 3. ОТЛАДКА ПРОГРАММНОГО МОДУЛЯ С ИСПОЛЬЗОВАНИЕМ СПЕЦИАЛИЗИРОВАННЫХ ПРО- ГРАММНЫХ СРЕДСТВ

3.1. Процесс отладки.

Отладка программного модуля проводилась с использованием встроенных средств Microsoft Visual Studio 2022. Отладка — это процесс обнаружения, локализации и исправления ошибок (багов) в программном коде.

Инструменты отладки:

- Точки останова (Breakpoints) - установка на критических строках кода для приостановки выполнения программы в нужном месте. Позволяет проверить состояние переменных на момент остановки.
- Пошаговое выполнение (Step Over, Step Into, Step Out) - построчное исполнение кода с возможностью входа внутрь методов (Step Into) или обхода их (Step Over). Используется для отслеживания логики выполнения.
- Окно Locals - автоматическое отображение всех локальных переменных текущего метода с их значениями и типами.
- Окно Watch - отслеживание значений выбранных выражений и переменных в реальном времени. Удобно для мониторинга ключевых данных.
- Окно Immediate Window - выполнение произвольных выражений и вызов методов в контексте текущей точки останова. Позволяет быстро проверить гипотезы.
- Окно Call Stack - отображение цепочки вызовов методов (стека вызовов), что помогает понять путь выполнения программы до текущей точки.

- Окно Output - вывод сообщений отладки, исключений, трассировки.

Используется для логирования операций.

При отладке метода `GetStatistics()` была установлена точка останова на строке возврата кортежа. В окне Locals было проверено, что переменные `completed`, `overdue` и `total` корректно вычисляются с помощью LINQ-метода `Count()`. В окне Watch отслеживалось значение `_tasks.Count` для контроля общего количества задач. Было выявлено, что при пустой коллекции метод возвращает корректные нулевые значения, что соответствует ожидаемому поведению.

РАЗДЕЛ 4. ОСУЩЕСТВЛЯТЬ РАЗРАБОТКУ ТЕСТОВЫХ НАБОРОВ И ТЕСТОВЫХ СЦЕНАРИЕВ ДЛЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

4.1. Процесс тестирования

Тестирование проводилось по методологии белого ящика с использованием модульных тестов. Цель — проверка корректности работы отдельных компонентов библиотеки TaskManager.Core.

Виды тестирования:

- Модульное тестирование (Unit Testing) - проверка изолированных методов TaskService;
- Интеграционное тестирование - проверка взаимодействия TaskService с JSON-хранилищем;
- Функциональное тестирование - проверка соответствия реализации требованиям ТЗ.

4.2. Создание unit-тестов

Создан проект TaskManager.Tests с использованием фреймворка MSTest. Реализованы 8 тестовых методов:

```
namespace TaskManager.Tests
{
    [TestClass]
    public class TaskServiceTests
    {
        private string _testFile = "test_tasks.json";

        [TestInitialize]
        public void Setup()
        {
            if (File.Exists(_testFile))
```

```

        File.Delete(_testFile);
    }

    [TestCleanup]
    public void Cleanup()
    {
        if (File.Exists(_testFile))
            File.Delete(_testFile);
    }

    [TestMethod]
    public void AddTask_ShouldAddTaskToList()
    {
        var service = new TaskService(_testFile);
        var task = service.AddTask("Test", "Desc", TaskPriority.High,
DateTime.Now.AddDays(1));

        Assert.AreNotEqual(Guid.Empty, task.Id);
        Assert.AreEqual("Test", task.Title);
        Assert.AreEqual(1, service.GetAllTasks().Count());
    }

    ...

    }
}

```

Результаты тестирования: все 8 тестов успешно пройдены (см. рис. 2).
 Покрывание кода: основные методы TaskService (CRUD, фильтрация, поиск, сортировка, статистика, сохранение/загрузка).

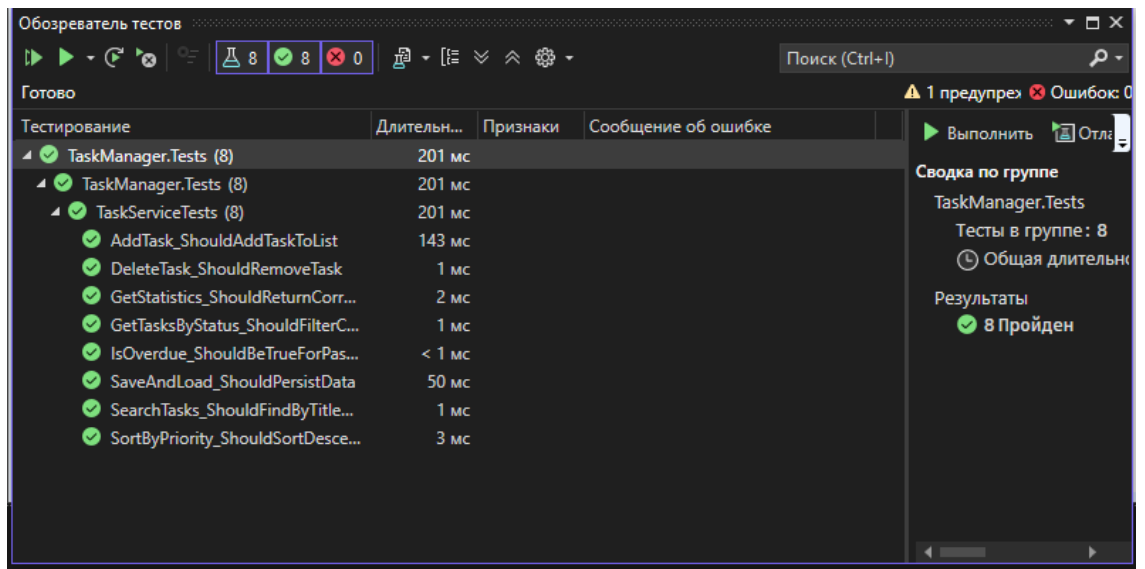


Рисунок 2. Выполнение тестов

РАЗДЕЛ 5. ИНСПЕКТИРОВАНИЕ КОМПОНЕНТ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ НА ПРЕДМЕТ СООТВЕТСТВИЯ СТАНДАРТАМ КОДИРОВАНИЯ

5.1. Процесс инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования.

Виды инспекции:

- Статический анализ кода - проверка без выполнения программы;
- Ручное ревью кода - проверка руководителем/коллегой;
- Автоматическая инспекция - с помощью инструментов Visual Studio.

Актуальные стандарты кодирования:

- Microsoft C# Coding Conventions - официальные соглашения Microsoft;
- StyleCop - инструмент анализа стиля кода (анализатор правил SA);
- SOLID-принципы - принципы объектно-ориентированного проектирования.

Программы для инспектирования:

- Visual Studio Code Analysis - встроенный анализатор;
- Roslyn Analyzers - компиляторные анализаторы;
- StyleCop.Analyzers - NuGet-пакет для проверки стиля.

Код соответствует основным стандартам кодирования C#. Рекомендуется добавить XML-документацию для всех публичных методов интерфейса ITaskService.

ЗАКЛЮЧЕНИЕ

В результате прохождения учебной практики с 8 по 28 июня 2026г. были закреплены теоретические знания и расширены профессиональные умения в области разработки программного обеспечения на языке C# с использованием платформы .NET Framework. Практика позволила применить на практике принципы объектно-ориентированного программирования, работу с коллекциями, LINQ-запросы, сериализацию данных, а также освоить методы модульного тестирования и отладки приложений.

По итогам практики получены следующие результаты:

1. Разработано техническое задание на создание приложения «Менеджер задач» в соответствии с требованиями ГОСТ 19.201-78 и ГОСТ 34.602-89. В ходе разработки ТЗ был проведён анализ предметной области, описаны бизнес-процессы «AS IS» и «TO BE», сформулированы функциональные и нефункциональные требования к программному продукту. Техническое задание стало основой для всех последующих этапов разработки.
2. Спроектирована и реализована модульная архитектура приложения с разделением на три проекта: библиотеку классов (TaskManager.Core), содержащую бизнес-логику; проект модульных тестов (TaskManager.Tests), обеспечивающий проверку корректности работы компонентов; и графический интерфейс (TaskManager.WinForms), реализующий взаимодействие с пользователем. Такое разделение соответствует принципам SOLID и позволяет легко расширять и модифицировать приложение без изменения существующего кода.
3. Реализована полная функциональность управления задачами в соответствии с требованиями технического задания: добавление новых задач с указанием названия, описания, приоритета, срока выполнения и статуса; просмотр списка всех задач в табличном виде; фильтрация задач по статусу (Новая, В процессе, Завершена); поиск задач по

названию или описанию с использованием LINQ; редактирование статуса существующих задач; удаление задач; сортировка по приоритету (убывание) и сроку выполнения (возрастание); отображение статистики в реальном времени (общее количество, завершённые, просроченные); визуальное выделение важных задач (жёлтым цветом и звёздочкой) и просроченных задач (красным цветом).

4. Обеспечено надёжное сохранение данных в JSON-формате с автоматической сериализацией и десериализацией при каждой операции модификации.
5. Разработаны и успешно пройдены 8 модульных тестов, покрывающих основную функциональность библиотеки TaskManager.Core: добавление задач, фильтрация по статусу, поиск по названию и описанию, удаление, сортировка по приоритету, получение статистики, сохранение и загрузка данных, проверка просроченности.
6. Проведено инспектирование кода на соответствие стандартам кодирования C# (Microsoft C# Coding Conventions) и SOLID-принципам объектно-ориентированного проектирования. Анализ показал, что код соответствует основным требованиям.
7. Освоены навыки работы с интегрированной средой разработки Microsoft Visual Studio 2022, включая использование дизайнера форм, отладчика с точками останова и пошаговым выполнением, тестового обозревателя, а также менеджера пакетов NuGet для подключения сторонних библиотек.
8. Получен практический опыт работы с системой контроля версий Git и размещения проектов на платформе GitHub, что является необходимым навыком для современного разработчика программного обеспечения.

По окончании практики был получен практический опыт разработки требований к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонентов;

интеграции модулей в программное обеспечение; отладки программного модуля с использованием специализированных программных средств; разработки тестовых наборов и тестовых сценариев для программного обеспечения; инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. ГОСТ 19.101-77. ЕСПД. Виды программ и программных документов. - М.: Изд-во стандартов, 1977.
2. ГОСТ 19.201-78. ЕСПД. Техническое задание. Требования к содержанию и оформлению. - М.: Изд-во стандартов, 1978.
3. ГОСТ 19.401-78. ЕСПД. Текст программы. Требования к содержанию и оформлению. - М.: Изд-во стандартов, 1978.
4. ГОСТ 19.404-79. ЕСПД. Пояснительная записка. Требования к содержанию и оформлению. - М.: Изд-во стандартов, 1979.
5. ГОСТ 34.602-89. Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. - М.: Изд-во стандартов, 1989.
6. Троелсен, Э. Язык программирования C# 7 и платформы .NET и .NET Core / Э. Троелсен. - 8-е изд. - СПб.: Символ-Плюс, 2018. - 1328 с.
7. Рихтер, Дж. CLR via C#. Программирование на платформе Microsoft .NET Framework 4.5 на языке C# / Дж. Рихтер. - 4-е изд. - СПб.: Символ-Плюс, 2013. - 896 с.
8. Microsoft. C# Coding Conventions. - URL: <https://docs.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/coding-conventions> (дата обращения: 15.06.2024).
9. Microsoft. .NET Framework 4.6 Documentation. - URL: <https://docs.microsoft.com/en-us/dotnet/framework/> (дата обращения: 10.06.2024).
10. Newtonsoft.Json Documentation. - URL: <https://www.newtonsoft.com/json/help/html/Introduction.htm> (дата обращения: 12.06.2024).

Требование к написанию программного модуля

Задание.

Цель:

Практика работы с классами, коллекциями, файловым вводом-выводом, LINQ и основами ООП в C#.

1. Основные требования: Разработать приложение для управления задачами с возможностью:

1.1 Добавления новой задачи (название, описание, приоритет, срок выполнения, статус).

1.2 Просмотра списка всех задач.

1.3 Фильтрации задач по статусу (например, "Новая", "В процессе", "Завершена").

1.4 Поиска задач по названию или описанию.

1.5 Редактирования существующих задач.

1.6 Удаления задач.

1.7 Сохранения и загрузки задач в/из файла (JSON или XML).

1.8 Основная логика приложения должна находиться в динамической библиотеке, а пользовательский интерфейс должен представлять из себя WPF-приложение.

1.9 Для классов в библиотеке необходимо написать модульные тесты.

2. Дополнительные задания (по желанию)

2.1 Реализовать сортировку задач по приоритету или сроку выполнения.

2.2 Добавить возможность пометки задачи как "Важная" (выводить звёздочкой или цветом в консоли).

2.3 Реализовать простую статистику (например, сколько задач завершено, сколько просрочено).

Сопроводительная записка должна содержать:

- титульный лист,

- оглавление,
- диаграмму классов,
- подробное описание действий по созданию ПО (с иллюстрациями).

Исходные коды приложения и сопроводительную записку выложить на GitHub и ссылку на репозиторий прислать мне на почту.